THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

Human Activity Recognition (HAR)

DATS 6303 – Deep Learning

Anusha Umashankar

Abstract

This project focuses on developing a Human Activity Recognition (HAR) system using the Long-term Recurrent Convolutional Network (LRCN) architecture to classify activities from video data. The system integrates Convolutional Neural Networks (CNN) for spatial feature extraction and Long-Short Term Memory (LSTM) networks for capturing temporal dependencies in videos. The final goal is to provide a real-time activity prediction model hosted on a Flask-based web application, allowing users to upload videos and receive predictions for human activities. The project uses the UCF50 dataset and employs PyTorch for model development and Flask for deployment.

1. Introduction

Human Activity Recognition (HAR) from video data is a computer vision problem, focusing on interpreting and understanding human actions through visual information captured in video sequences. This task is inherently challenging due to the dynamic nature of human movements, the vast diversity of possible activities, and the variability of the environments in which these activities take place.

With complex neural networks significant improvements have been made in the past decade. These improvements have paved the way for HAR's application in a wide array of fields such as surveillance, sports analytics, and human-computer interaction (HCI).

1. *Surveillance*: In the context of security and monitoring, HAR systems can analyze video footage in real-time to automatically detect suspicious activities, such as fights, falls, or unauthorized access. By identifying these behaviors quickly, HAR systems can enhance security and provide early alerts to prevent potential threats.
2. *Sports Analytics*: HAR plays an important role in sports by enabling detailed performance analysis of players. It can be used to identify specific actions (such as a basketball jump shot or a football pass) and help coaches and analysts gain deeper insights into game strategies, player performance, and injury prevention.
3. *Human-Computer Interaction (HCI)*: HAR can also be used to improve human-computer interactions by enabling devices to respond to human gestures and actions. This leads to more intuitive interfaces, where users can interact with applications without the need for traditional input devices like keyboards.

This project aims to tackle the complexities involved in recognizing human activities from video data using the UCF50 dataset, a popular benchmark in the field of HAR. The proposed solution integrates two advanced deep learning techniques: CNNs for spatial feature extraction and LSTMs for modeling the temporal dependencies in video sequences. By combining these two powerful approaches, this system is designed to efficiently recognize and classify a variety of human activities in real-time.

Additionally, the model is incorporated into a user-friendly web application built with Flask, allowing users to upload videos and receive real-time predictions of human activities directly from the application interface.

2. Problem Statement

The problem is to accurately classify human activities from video data. Traditional image classifiers like CNNs can only analyze individual frames, while LSTMs are designed to capture sequential patterns in data. Combining CNNs and LSTMs allows the model to process both spatial and temporal features of videos. The challenge lies in developing an architecture that can effectively handle both spatial and temporal features, ensuring robust classification in real-time applications.

3. Related Work

Action recognition has been a research area in computer vision, with a lot of efforts focusing on constrained datasets such as KTH, Weizmann, and IXMAS, which have limited categories. Many older approaches like finite state models and hidden Markov models may be effective on small datasets, but they fail to handle large datasets due to their limitation in understanding camera motion, background clutter and many more.

Parallel to these researches, deep learning has introduced several strategies for video classification. One basic approach is **single-frame classification**, where each video frame is classified independently using an image classifier such as a CNN. While simple, this method fails to capture the temporal dynamics of actions, often misclassifying (e.g., labeling backflip as fall). Averaging across frame predictions can offer minor improvements, but limits accuracy due to lack of temporal modeling.

Late fusion improves upon this by aggregating predictions from individual frames at a later stage, allowing for some temporal reasoning. However, it remains limited in

performance, especially for complex or long-duration activities. **Early fusion** approaches attempt to combine spatial features across frames early in the pipeline, often using stacked frame inputs, which improves temporal understanding but still lacks dynamic modeling over time.

3D Convolutional Neural Networks (3D CNNs) can be used to extract spatial and temporal features jointly across frame sequences. While this approach is powerful, 3D CNNs are computationally expensive and often impractical for large-scale datasets. Another approach is **pose estimation followed by sequence modeling**, where human landmarks are extracted per frame and fed into an LSTM network to model temporal dynamics. This method can work well but doesn't capture important contextual information, such as scene and object information.

Long Short-Term Memory (LSTM) networks, a type of Recurrent Neural Network (RNN), are suited for modeling sequential data and have been applied to action recognition tasks to capture long-term dependencies in videos. To leverage both spatial and temporal aspects effectively, in this project a combination of CNNs for spatial feature extraction with LSTMs for temporal modeling is implemented. This **CNN-LSTM architecture** has shown promising results, especially to UCF50 dataset, which are composed of real-world web videos with significant variability.

4. Solution and Methodology

The proposed solution employs an LRCN architecture, combining CNNs and LSTMs. The CNN is used to extract spatial features from each frame of the video, and the LSTM captures the temporal relationships across frames to predict the activity.

- CNN: Extracts spatial features from individual frames using a series of convolutional layers.
- LSTM: Models the temporal dependencies between frames by processing sequences of feature maps produced by the CNN.

- Flask Web Application: The model is deployed as a Flask-based web application, where users can upload videos and receive predictions.

The architecture was implemented in PyTorch, and the UCF50 dataset was used for training and evaluation. The model is fine-tuned for both spatial and temporal accuracy.

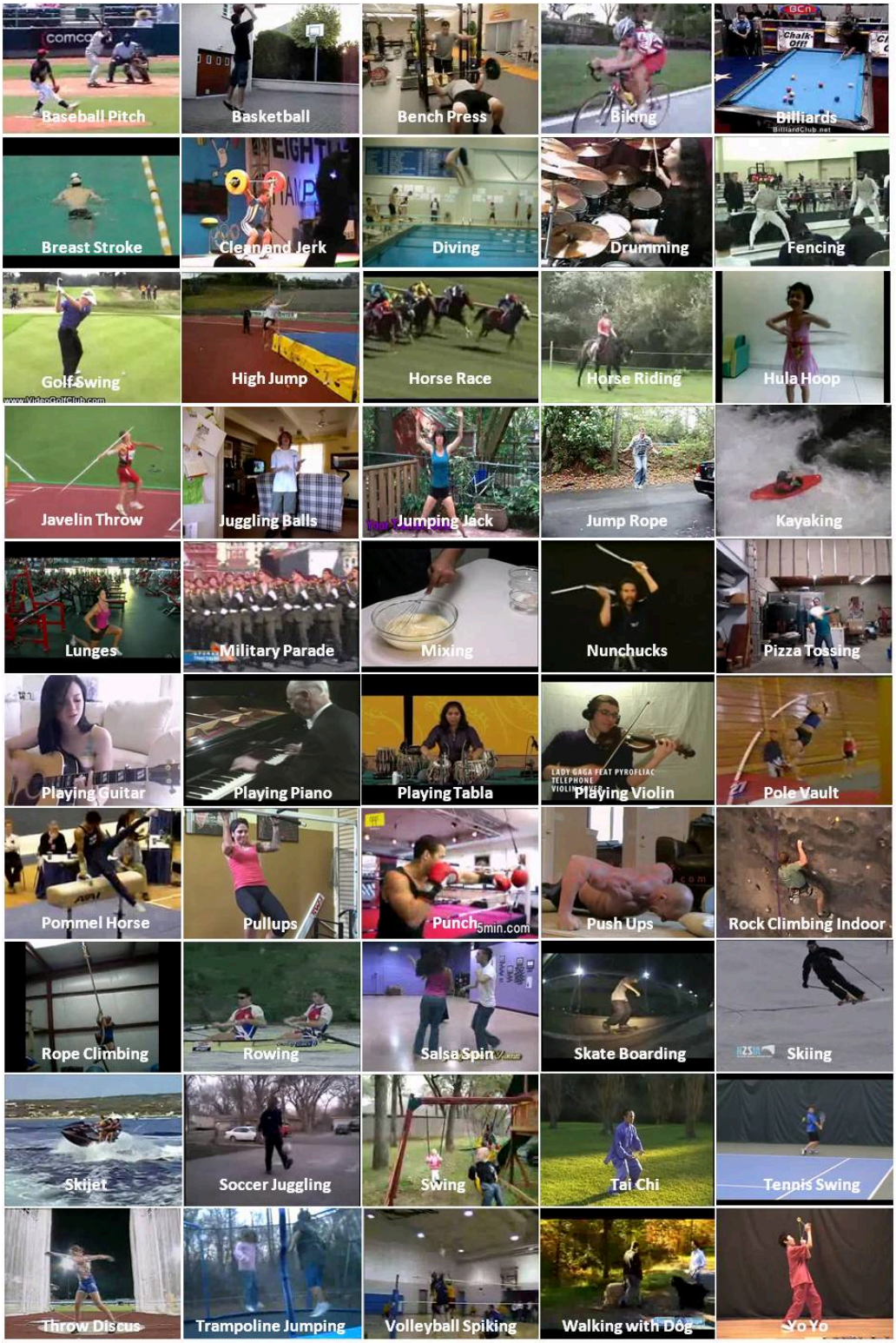
5. Results and Discussion

This section presents the results from the training and testing of the HAR model. Metrics, such as accuracy and confusion matrices, were used to evaluate the model's performance.

5.1 Experimentation protocol

The model is trained on the UCF50 dataset, which contains 50 action categories. The training was performed using the Adam optimizer, with a learning rate of 0.0001. Data augmentation techniques, including random cropping, flipping etc.., were applied to enhance the model's robustness.

The 50 action categories are :



5.1.1 Data Preprocessing

The video data is loaded using the Dataset class (*VideoDataset)* in which each video is read using OpenCV (cv2.VideoCapture). A fixed number of frames (SEQUENCE_LENGTH=20) are sampled evenly across the entire video. Each frame is resized to (IMAGE_HEIGHT = 64, IMAGE_WIDTH = 64) using *PIL* and transformed into tensors with *torchvision.transforms* . Data augmentations like random horizontal flips and rotations are applied to simulate variations. The dataset returns a tensor of shape [T, C, H, W] (time, channels, height, width) and a label index.

5.1.2 Model Architecture

The architecture begins with a CNN feature extractor consisting of two convolutional layers: the first with 32 filters and the second with 64 filters. These convolutional layers are followed by ReLU activation and batch normalization. A max-pooling layer is implemented after these convolutional layers. The output from the CNN is then reshaped into a sequence of feature vectors and passed to an LSTM layer with 128 hidden units. After that, a fully connected (FC) layer is used to map the final output of the LSTM to the class predictions. This FC layer has a size of 128 to the number of classes in the dataset. The model leverages dropout in the CNN layers to prevent overfitting. The final output is a predicted label for the entire video.

5.1.3 Training Loop

For each epoch, the model iterates over batches of video data with shape [B, T, C, H, W] (batch, time, channels, height, width), computing predictions and loss using *CrossEntropyLoss*. Backpropagation and an optimizer step follow, while tracking accuracy and loss. After each epoch, the model is evaluated on the test set using *evaluate_model()*. The dataset is split into 70% for training, 10% for validation, and 20% for testing, with data wrapped in PyTorch DataLoaders that apply batching and shuffling to the training set. The model with the best *test accuracy* is saved with a timestamp using *torch.save()*.

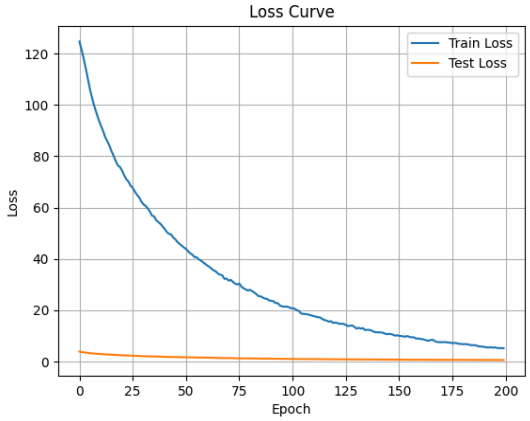
5.1.4 Prediction

Similar to training, *SEQUENCE_LENGTH(=20)* frames are read from the video, resized, and transformed into tensors, resulting in a shape of [1, T, C, H, W]. The LRCN model is loaded using *torch.load()* to restore its weights, and if no path is provided, it searches *code/maincode/models/* for the latest model. During inference, the transformed video tensor is passed through the model, and the predicted class index is obtained using *argmax*, which is then mapped to a class label.

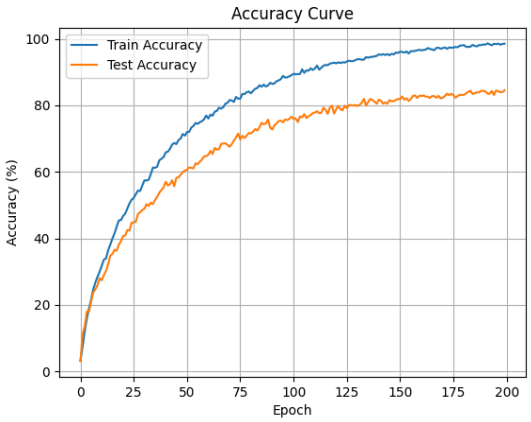
5.2 Results

The model demonstrates strong learning capabilities with a high training accuracy of 98.85% and achieving 84.43% validation and 83.77% test accuracy on real-world video data. These results reflect a well-performing system with potential for further optimization.

The loss curve:



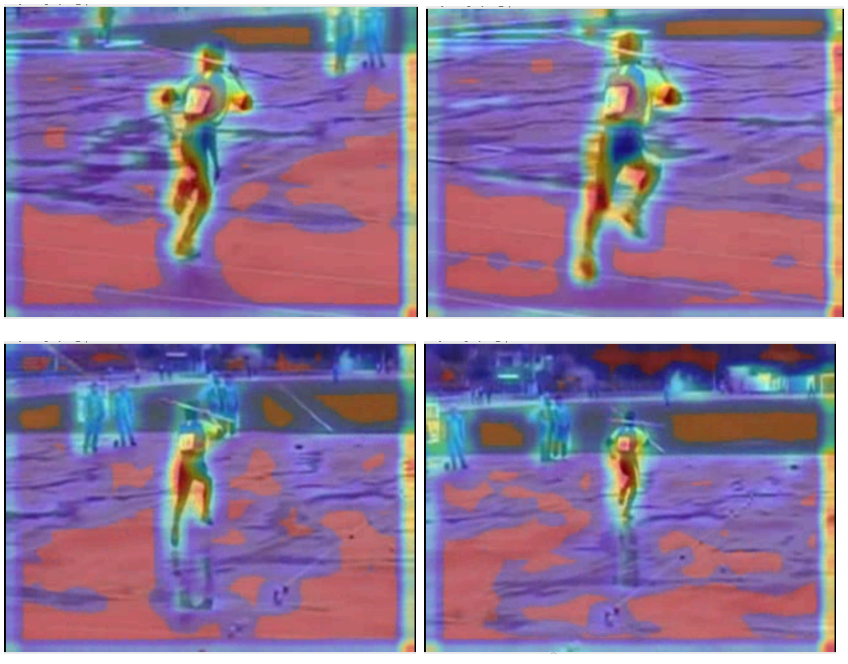
The accuracy curve:



5.3 Post-training analysis

Post Training analysis is done by generating a "heatmap". For heatmap generation, a forward hook captures CNN feature maps, which are averaged, resized, and overlaid on the original frames with a color map. The outputs are saved in the *analysis* folder.

The Heatmap for Javelin Throw: https://youtu.be/O9gfeRcOvqk



6. Discussion

The results demonstrate that the CNN+LSTM model achieves a high level of accuracy (test accuracy being 83.77% and validation accuracy of 84.43%) in recognizing human activities. However, the model's performance could be further enhanced by increasing the dataset size and designing a more complex model architecture. The model struggled slightly with classifying activities with similar visual cues, such as "Baseball" and "Jump Rope." Future work could explore incorporating additional features like environmental context (e.g., background and objects) to improve classification.

7. Conclusion

The Human Activity Recognition system developed in this project demonstrates a solid understanding of both spatial and temporal feature extraction. The use of CNNs for spatial analysis and LSTMs for temporal sequence modeling results in an effective model capable of real-time predictions. The Flask web application provides an interactive platform for users to test the system. Moving forward, expanding the model's capability to handle more diverse video inputs and integrating contextual features could further improve accuracy. Also complex architecture like vision transformers could outperform this architecture.

8. Bibliography

[1] Reddy, K.K., Shah, M. Recognizing 50 human action categories of web videos. *Machine Vision and Applications* 24, 971–981 (2013). https://doi.org/10.1007/s00138-012-0450-4

[2] P. M., Naidu, R., & P., A. (2024). Combining deep learning techniques for enhanced human activity recognition: A hybrid CNN-LSTM fusion approach. In 2024 IEEE International Conference on Contemporary Computing and Communications (InC4) (pp. 1–7). IEEE. https://doi.org/10.1109/InC460750.2024.10649335